

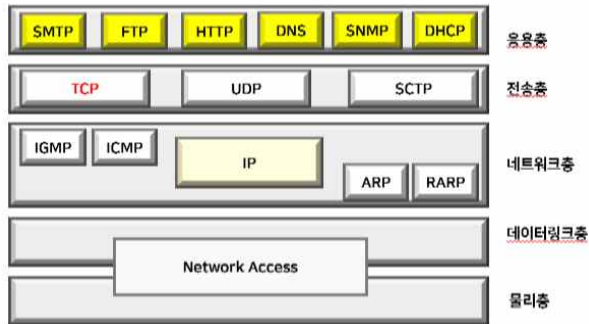
11주차 - 2차시 전송 계층 프로토콜 TCP

I. TCP 서비스

1. TCP 개요

1) 개요

- TCP, UDP, SCTP
- TCP/IP 프로토콜 모델 계층에서 TCP 위치



2) TCP 프로토콜의 역할과 기능

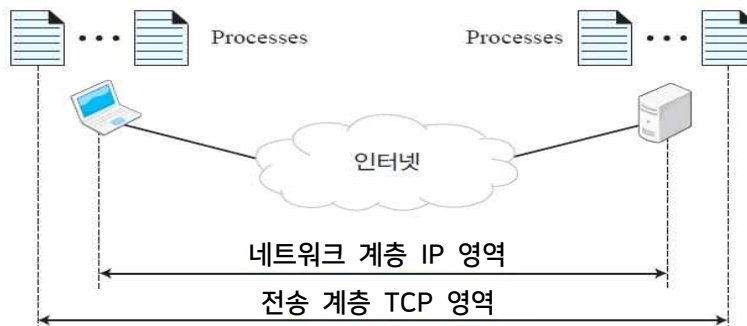
프로세스간 통신 설정	▪ 포트 번호를 이용
전송 단계에서 흐름 제어 메커니즘 제공	▪ 슬라이딩 윈도우 프로토콜 이용
전송 단계에서 오류 제어 메커니즘 제공	▪ 응답 패킷, 시간-초과, 재전송 방식 이용

- 연결 지향 형태의 신뢰성 있는 프로토콜이다.
- TCP는 연결형 프로토콜
- 물리적인 연결이 아닌 가상 연결
- 연결 설정 → 데이터 교환 → 해제 순으로 진행

2. TCP 연결형 데이터 서비스

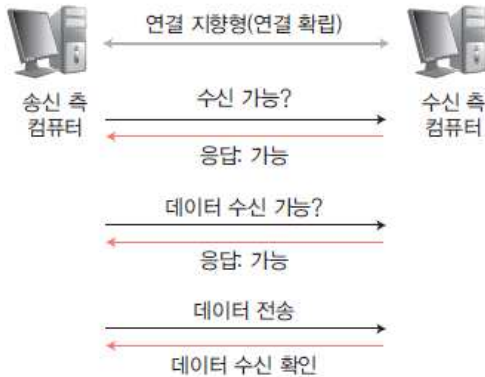
1) TCP와 IP 역할 비교

- IP는 네트워크 레벨에서의 통신 기능을 담당, 즉 메시지를 발신지에서 목적지 컴퓨터까지 전송한다.
- TCP는 적절한 응용 프로그램으로 메시지를 전송하는 역할을 한다.



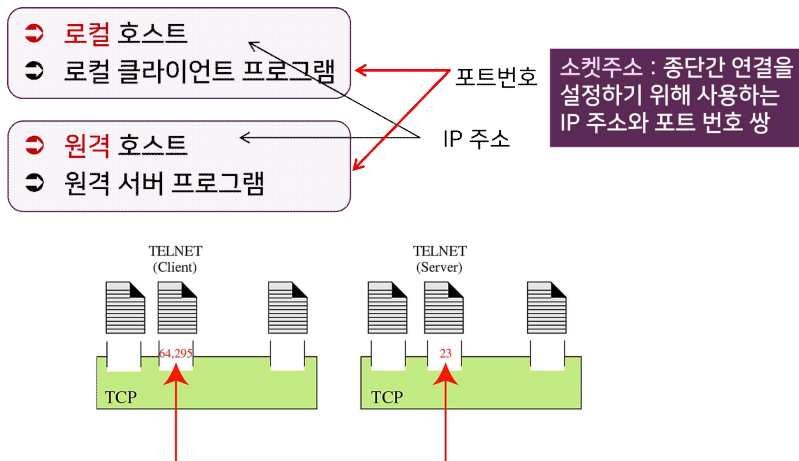
2) TCP 연결 지향형 프로토콜

- 송신측 컴퓨터와 수신측 컴퓨터가 데이터를 전송하기 전에 먼저 데이터를 송수신할 수 있는 연결 통로를 만들고 데이터를 전송하는 프로토콜이다.
- 연결 지향형 프로토콜은 신뢰성있는 데이터 전송을 보장하며, 오류가 발생하면 수신측에 알려준다.



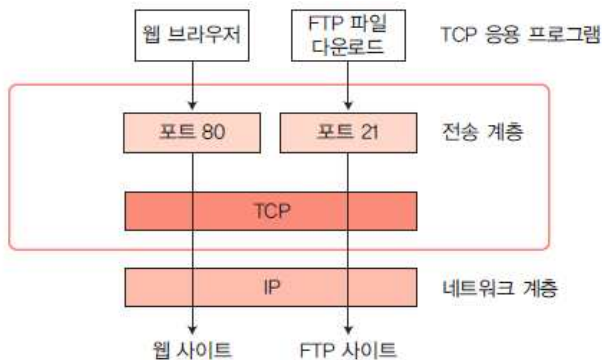
3. TCP 포트 번호

1) 클라이언트-서버 통신에 필요한 사항

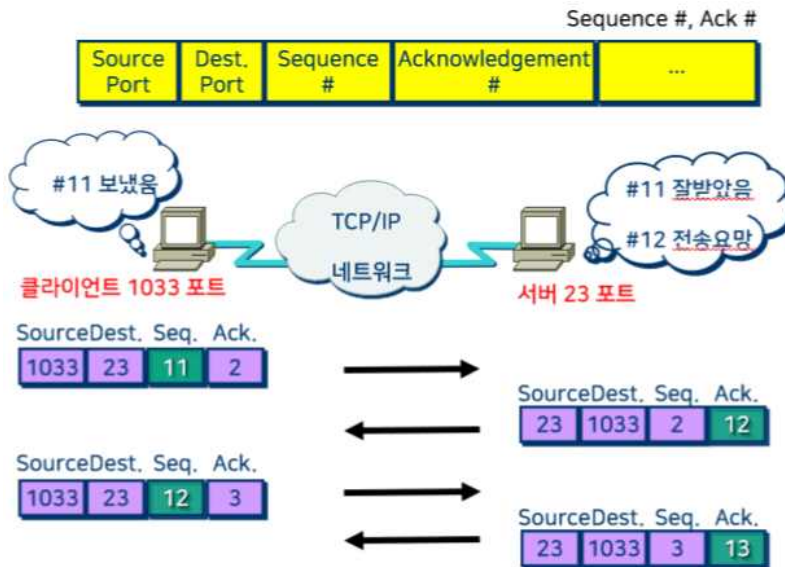


2) 프로세스 대 프로세스 통신

- 포트는 TCP가 상위 계층으로 데이터를 전달하거나 상위 계층에서 TCP로 데이터를 전달할 때 이용하는 데이터의 이동 통로를 말한다.
- TCP 프로토콜이 여러 개의 포트를 사용해서 상위 계층의 프로그램과 각각 통신이 가능하다.



3) TCP 포트 번호를 이용한 통신



4) TCP에서 사용되는 잘 알려진 포트 번호

- 어떤 애플리케이션이 사용되고 있는지 구분하려면 포트 번호가 필요
- 웹 서비스는 HTTP 프로토콜을 사용하므로 포트 번호가 80번이다.
- 웹 브라우저를 사용하여 서버에 접속할 때 포트 번호를 입력하지 않아도 자동으로 80번이 할당된다.

포트 번호	애플리케이션	포트 번호	애플리케이션
21	FTP	80	HTTP
25	SMTP	110	POP3
53	DNS	443	HTTPS

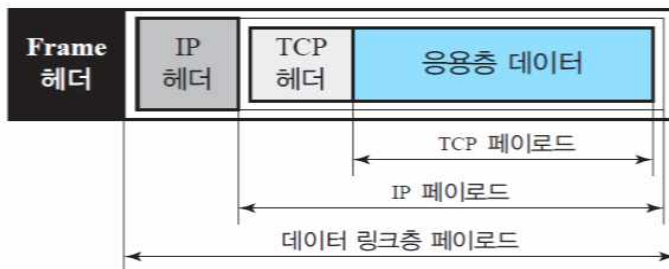
※ TCP에서 사용하는 잘 알려진 포트 번호

포트	프로토콜	설명
7	Echo	Echoes a received datagram back to the sender
9	Discard	Discards any datagram that is received
11	Users	Active users
13	Daytime	Returns the date and the time
17	Quote	Returns a quote of the day
19	Chargen	Returns a string of characters
20과 21	FTP	File Transfer Protocol (Data and Control)
23	TELNET	Terminal Network
25	SMTP	Simple Mail Transfer Protocol
53	DNS	Domain Name Server
67	BOOTP	Bootstrap Protocol
79	Finger	Finger
80	HTTP	Hypertext Transfer Protocol

II. TCP 세그먼트

1. TCP 세그먼트(Segment)

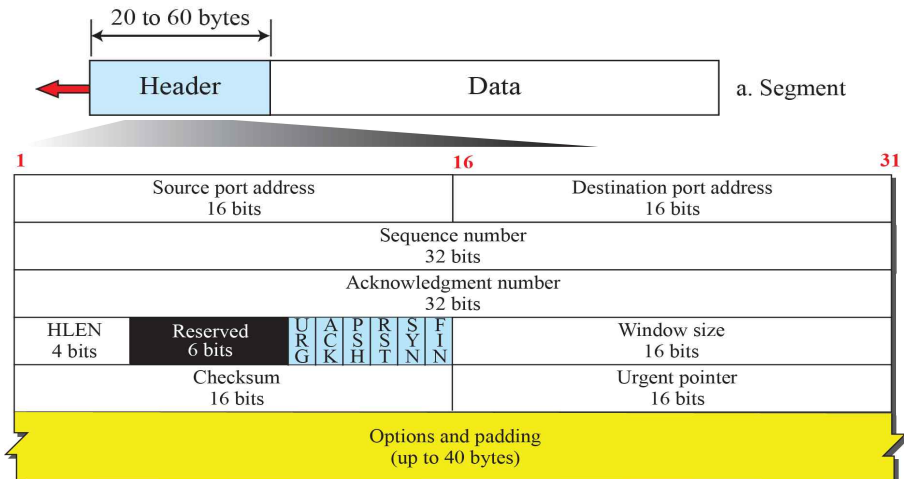
- 세그먼트는 TCP를 이용하여 두 장치 간에 전달하는 데이터의 단위이다.
- TCP 세그먼트는 20 ~ 60 바이트의 TCP 헤더와 응용 프로그램으로부터 발생하는 TCP 데이터(페이로드)로 구성된다.
- TCP 세그먼트는 IP 데이터그램에 캡슐화되고, 다시 데이터 링크층의 프레임에 캡슐화된다.



2. TCP 헤더 구조

1) TCP 헤더 형식

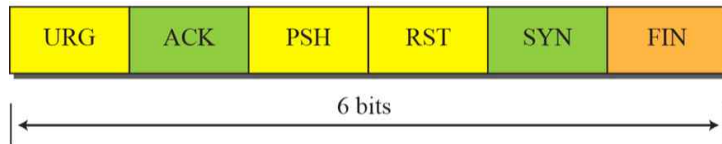
- 헤더는 기본이 20 바이트이고, 옵션을 포함하는 경우 최대 60 바이트



2) TCP 헤더 필드

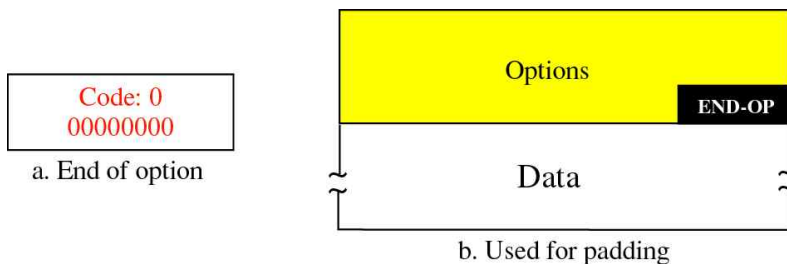
- 송신지 포트 번호(Source Port Address) : 16비트
 - 세그먼트를 전송하는 호스트에 있는 응용 프로그램의 포트 번호
- 수신지 포트 번호(Destination Port Address) : 16비트
 - 수신지 호스트에서 수행하는 프로세스가 사용하는 포트
- 순서 번호(Sequence Number) : 32비트
 - TCP 세그먼트에 포함된 데이터의 첫 번째 바이트에 부여된 번호
 - 0부터 시작해서 $2^{32}-1$ 을 초과하면 다시 0으로 되돌아온다.
- 확인 응답 번호(Acknowledgement Number)
 - 세그먼트를 수신한 노드가 송신측으로부터 수신하려는 바이트 번호
 - 수신한 마지막 바이트의 순서 번호 + 1이다.
- 헤더 길이(Header length) : 4비트
 - TCP 헤더 길이를 4바이트 워드 값으로 나타내며,

- 필드 값은 $5 \times 4 = 20$ 에서 $15 \times 4 = 60$ 사이의 값이 될 수 있다.
- 윈도우 크기(Window Size) : 16비트
 - 상대방이 유지해야 하는 바이트 단위의 윈도우 크기를 정의.
- 제어 필드(Control) : 코드 비트 6개로 구성
 - 연결 제어 정보가 기록되는 Code Bit는 각 비트 역할이 정해져 있으며, 초기값이 0이고 비트가 활성화 되면 1이 된다.



코드 비트	설명
URG	긴급하게 처리할 데이터가 들어 있음
ACK	응답 확인 번호 사용
PSH	TCP가 받은 데이터를 상위 계층에 전달
RST	연결 재설정
SYN	연결을 초기화 하려고 순서 번호 동기화
FIN	데이터 송신 종료

- 검사합 : 16비트
 - 헤더 부분의 오류를 검출하는 검사합 계산이 포함된다.
 - UDP 데이터그램에 있는 검사합은 선택사항이지만 TCP에서는 필수로 검사하는 사항이다.
- 긴급 포인터(urgent pointer) : 16비트
 - 세그먼트가 긴급 데이터를 포함하고 있을 때 사용되며, URG 코드비트가 1로 설정되어 있는 경우에만 유효하다.
- 옵션 필드(option)
 - 옵션 선택 사항 필드가 지정되지 않았다면 TCP 헤더의 크기는 20byte 이다.
 - 최대 40 바이트 까지의 옵션 정보가 있을 수 있다.
 - 옵션-끝(End of option)은 맨 끝에 더 이상의 옵션이 없음을 나타내는 한 바이트 값



Ⅲ. TCP의 주요 기능

1. TCP 연결 설정

1) 3-way 핸드셰이킹

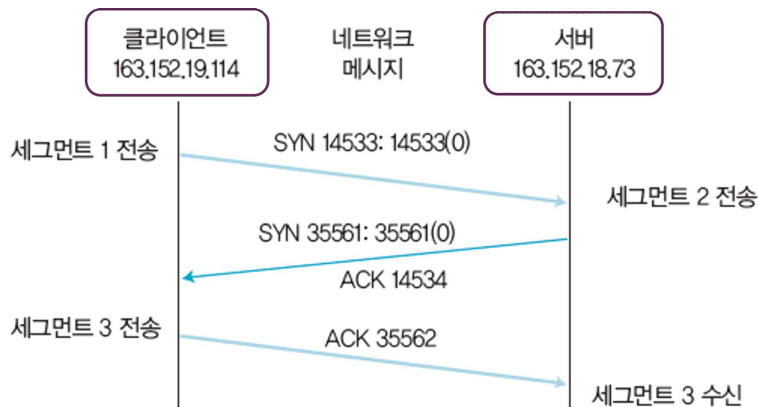
- 데이터를 전송하기 전에 연결을 확립하기 위해 패킷 요청을 세 번 교환하는 것을 3-way 핸드셰이킹이라고 한다.
- 3-way 핸드셰이킹은 상대방을 확인하고 약속을 하는 것처럼 네트워크에서 확실하게 데이터가 전송되는지 확인하면서 이루어지는 통신 방식이다.

2) 연결 설정 단계

- ① 송신 호스트는 수신측에게 연결 설정 세그먼트 초기화 정보 SYN 전송.
- ② 수신 호스트는 송신측에게 확인 응답 세그먼트 ACK 및 초기화 정보 세그먼트 SYN 전송
- ③ 송신 호스트는 수신 호스트에게 확인 응답 세그먼트 ACK 전송.



- SYN과 ACK를 사용한 3-way 핸드셰이킹 과정



3) 연결 종료 단계

- ① 송신 컴퓨터는 수신 컴퓨터로 연결 종료 요청 FIN을 보낸다.
- ②, ③ 수신 컴퓨터는 송신 컴퓨터로 연결 종료 확인 응답 ACK 및 연결 종료 요청 FIN을 보낸다.
- ④ 송신 컴퓨터는 수신 컴퓨터로 연결 종료 응답 ACK를 전송한다.

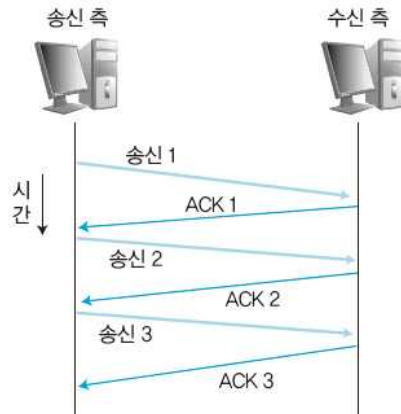


4) 연결 재설정

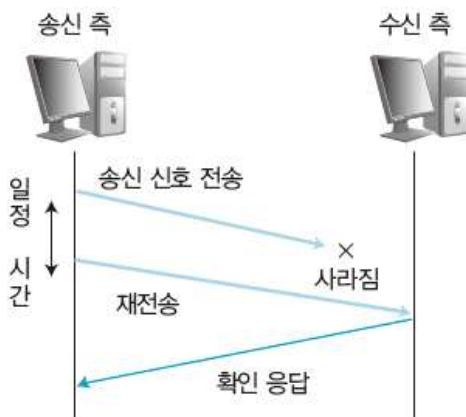
- 한쪽 편에 있는 TCP는 RST(reset) 플래그 비트를 1로 설정하여 전송 함으로써 연결 재설정을 한다.
- 연결 요청을 거절하거나, 중단하거나, 휴지상태에 있는 연결을 종료 할 수 있다.

5) 확인 응답

- 수신 측에서 데이터를 수신했을 때 이 사실을 송신 측에 통보하는 것으로, 핸드셰이킹이 해당된다.
- 확인 응답을 ACK(ACKnowledgement)라고 하는데, 확인 응답이 있어야 다음 데이터를 전송할 수 있다.



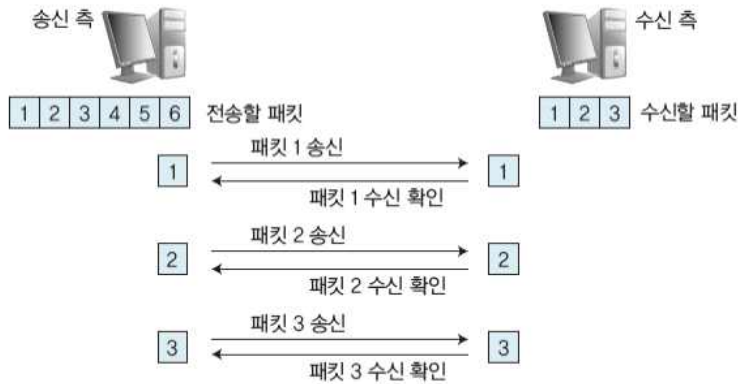
- 수신 측에서 데이터를 수신한 후에도 네트워크 문제로 데이터를 재전송하는 경우가 발생하는데,
- 이때는 이미 수신한 데이터이므로 재전송한 데이터는 폐기해야 되는데, 순서 번호를 설정하여 구분할 수 있다.



2. TCP 데이터 흐름 제어

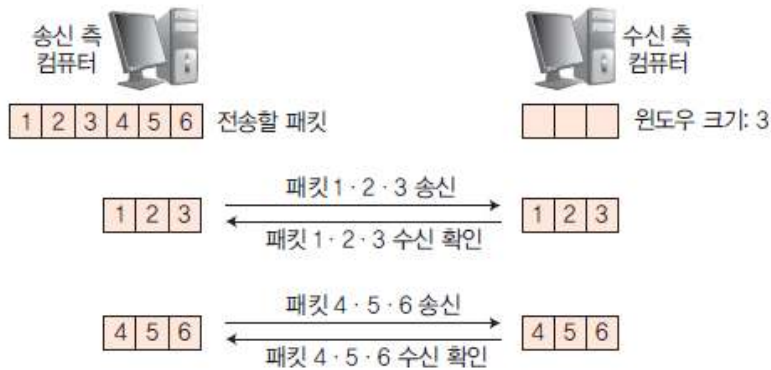
1) TCP 데이터 전송

- TCP에서는 데이터를 패킷이라는 작은 단위로 분할하여 전송하므로 수신 측 컴퓨터는 패킷을 재결합하여 다시 원래의 데이터로 만들어야 한다.
- 패킷을 전송할 때 네트워크 상황에 따라 패킷 도착 순서가 바뀔 수도 있다.
- 한 번에 하나씩 송수신하는 것보다는, 여러 패킷을 한 번에 송수신한 후 수신 확인 신호를 전송하는 것이 훨씬 효율적이다.



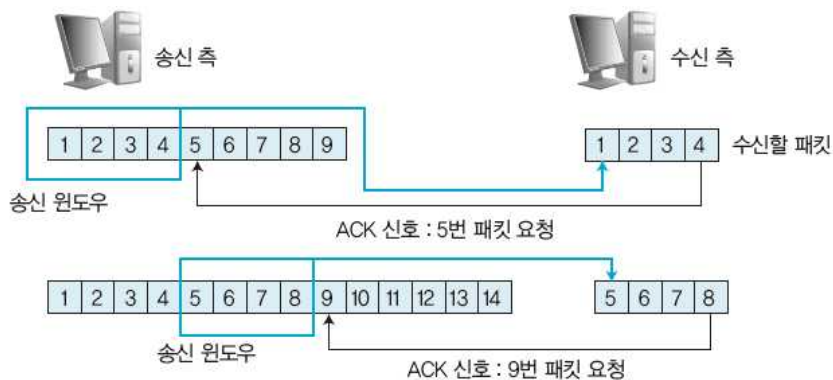
2) 윈도우 크기

- 한번에 전송할 수 있는 전체 패킷의 개수를 'TCP 윈도우 크기'라고 하는데, 윈도우 크기가 크면 한 번에 여러 패킷을 전송할 수 있다.
- 송수신 측 컴퓨터가 상대방의 윈도우 크기를 확인함으로써 오버플로가 발생하지 않게 데이터를 전송할 수 있다.



3) 슬라이딩 윈도우

- 송신측 버퍼 역할을 하기 위해서 송신 윈도우를 이동하는 방식이다.
- 수신측의 윈도우 크기가 4이므로 송신측은 1,2,3,4번 패킷을 전송,
- 수신 측 컴퓨터로부터 수신 확인 ACK 신호를 수신하면, ACK 신호에서 요청한 5번 패킷 위치로 송신 윈도우를 옮긴다.
- 다시 윈도우 안의 4개 패킷을 전송하고,
- 수신 측 컴퓨터로부터 ACK 신호를 수신하면 ACK 신호에서 요청한 9번 패킷 위치로 송신 윈도우를 옮긴다.



4) 윈도우 신드롬

(1) 어리석은 윈도우 신드롬

- 송/수신 응용 프로그램이 데이터를 천천히 생성하거나 천천히 처리할 때 발생한다.
- 예를 들어 1 바이트 데이터를 보내는 경우
 - 오버헤드는 41/1로 매우 비효율적이다.

(2) 송신 측에서 발생하는 신드롬 해결 방법

- 가능한 1바이트 데이터를 전송하지 못하게 한다.
- 데이터를 취합하여 큰 블록 데이터로 만들어 전송
- Nagle 알고리즘 적용
 - 송신 TCP는 응용층 프로그램으로부터 수신한 데이터를 첫 번째 세그먼트로 만들어 전송한다.
 - 첫 번째 세그먼트를 전송한 후, 송신 TCP는 수신 TCP로부터 확인 응답을 수신하거나, 최대 크기의 세그먼트를 구성할 정도로 데이터가 출력 버퍼에 저장되면 세그먼트를 전송한다.
 - 2번째 단계를 계속 반복한다.

(3) 수신 측에서 발생하는 신드롬 해결 방법

- 어리석은 창 신드롬이 발생하는 상황에서 한 바이트 수신 처리 후에 창 크기를 통보하게 되는 경우에 해결하는 방법
- Clark 해결 방법
 - 충분한 공간이 생기거나 적어도 버퍼가 반 이상 비어 있을 때까지 창 크기를 0으로 통일
- 지연된 확인 응답
 - 수신 버퍼가 충분한 공간이 생길 때까지 확인 응답 보류

5) 오류 발견과 정정

(1) 오류 발견 도구 : 검사합, 확인응답, 시간-초과(time out)

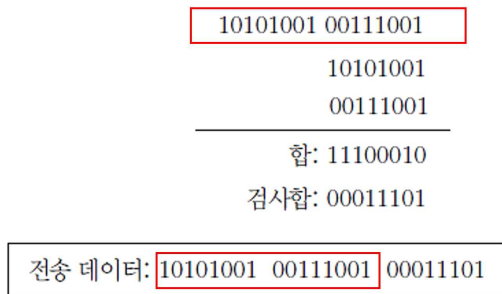
- TCP 세그먼트의 검사합 필드를 이용하여 훼손 여부 확인
- 수신을 송신측에 알려주는 확인 응답 이용한다.
- 시간-초과 전까지 확인 응답 되지 않으면 훼손 또는 손실 간주
- 오류 정정 메카니즘으로 시간-초과 카운터를 이용할 경우, 만일 카운터가 설정값에 다다르면 손상 혹은 손실로 판단하고 재전송

(2) TCP 검사합

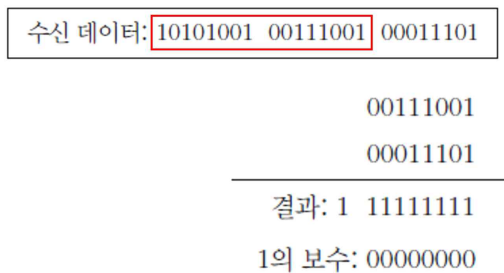
- 헤더 부분의 오류를 검출하는 검사합 계산이 포함된다.

송신 측	• 데이터 단위를 각각 n(보통 16)비트씩 섹션 m개로 나눈다. • 모든 섹션은 합을 만들려고 1의 보수를 사용하여 서로 더한다. • 합은 보수화되어 검사합이 된다. • 검사합의 값을 데이터와 함께 보낸다.
수신 측	• 데이터를 수신하고 나서 각각 n비트씩 섹션 m개로 나눈다. • 모든 섹션은 합을 만들려고 1의 보수를 사용하여 서로 더한다. • 합은 보수화되고 결과가 0이면 오류가 없는 것이고, 그렇지 않으면 오류가 발생한 것이다.

- [검사합 예] 16비트 블록을 8비트 검사합을 사용하여 전송하는 경우를 생각해보자.

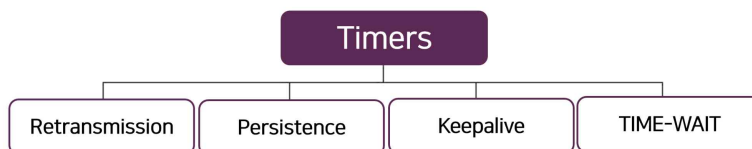


- 수신 데이터 섹션 3개를 모두 더한 후 1의 보수가 0인지 확인하여 결과가 0이면 오류가 없는 것



3. TCP 타이머

- TCP의 동작을 순조롭게 수행하기 위하여 4가지의 타이머를 사용한다.
- 재전송(retransmission), 지속(persistence), 킵얼라이브(keepalive), 시간-기다림(TIME-WAIT)을 이용한다.



1) 재전송 타이머(Retransmission Timer)

- 손실 또는 손상되어 버려지는 세그먼트를 처리
- 재전송 시간은 한 세그먼트를 전송하고 확인 응답을 기다리는 시간인 재전송 타임-아웃(RTO: Retransmission Time Out) 값으로 설정
- TCP는 세그먼트를 전송한 후, 해당 세그먼트를 위한 재전송 타이머를 구동하게 된다.
- 다음과 같은 두 가지 상황이 일어날 수 있음
 - 타이머가 끝나기 전에 특정 세그먼트에 대한 확인 응답이 수신되면, 그 타이머는 소멸
 - 만일 응답이 수신되기 전에 타이머가 종료되면 해당 세그먼트는 재전송되고 타이머는 초기화

2) 영속 타이머(persistence timer)

- 양쪽 TCP가 서로 기다리는 교착상태 발생으로 이를 해결하기 위하여 송신 TCP는 영속 타이머를 사용한다.
- 창 크기가 0인 경우를 처리하기 위한 타이머로서 수신 TCP가 창 크기 0을 통보한다.
 - 송신 TCP는 수신 TCP로부터 0 아닌 창 크기를 알리는 확인 응답을 수신하기까지는 세그먼트 전송 보류

- 만일 수신 TCP가 이런 확인 응답을 보냈지만 손실이 되면 송신 TCP는 기다리기만 하고,
- 수신 TCP는 송신 TCP가 세그먼트를 전송하기를 기다리기를 계속.

3) 킵얼라이브 타이머 (Keepalive Timer)

- 두 TCP 간에 설정된 연결이 오랫동안 휴지(idle) 상태에 있는 것을 방지하기 위한 타이머
- 시간-종료 2시간이 지나도록 클라이언트로부터 세그먼트를 수신하지 못하면 75초 간격으로 10개의 프로브(probe) 세그먼트를 전송하고,
- 응답이 없으면 다운으로 간주하고 연결을 종료한다.

4) 시간-대기 타이머 (Time-waited timer)

- 연결 종료 동안에 사용한다.